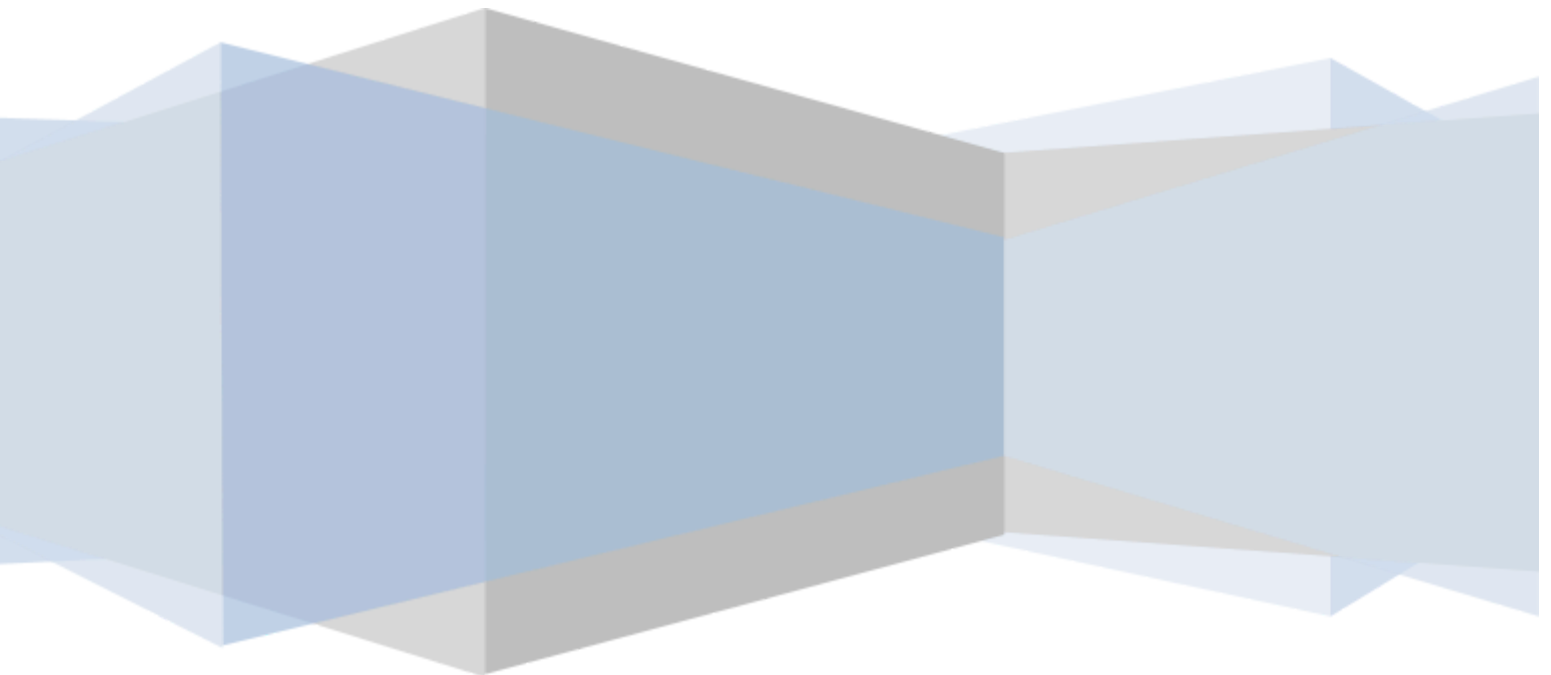


Java 8 추가 기능



INDEX

- 함수형 프로그래밍
- Functional Interface
- Stream API
- Date와 Time API

함수형 프로그래밍

$$\text{Program}(x) = f \circ g \circ \dots (x)$$

- 함수형 프로그래밍 언어를 사용해야 하는 이유

1. 높은 표현력을 통해 불필요한 코드를 줄일 수 있다.

(무엇을 하는지를 명확히 알 수 있다.)

2. 함수형 프로그래밍 언어군은 프로그래밍 언어론의 최신 연구 결과를 반영하고 있다.

함수형 인터페이스

- 함수형 인터페이스 (Functional Interface)

- **default method** 또는 **static method** 는 여러개 존재해도 상관없이 **추상 메소드를 오직 하나만 가지고 있는 인터페이스**를 말한다.
- **Java8**부터 인터페이스는 기본 구현체를 포함한 **default** 메소드를 포함할 수 있다.
- **@FunctionalInterface** 애노테이션을 가지고 있는 인터페이스이다.

(인터페이스 검증과 유지보수를 위해 사용 권장)

- 함수형 인터페이스 생성

```
@FunctionalInterface
interface CustomerInterface<T> {

    // 추상 메소드

    T myCall();

    //default method

    default void printDefault() {

        System.out.println("Hello Default!!");

    }

    // static method

    static void printStatic() {

        System.out.println("Hello Static!!");

    }

}
```

```
public class App {  
    Run | Debug  
    public static void main(String[] args) throws Exception {  
  
        //자바의 람다 표현식을 사용하여 CustomerInterface 인터페이스를 구현합니다.  
        CustomerInterface<String> func = () -> "Hello, World";  
  
        //메소드 호출 및 출력  
        System.out.println(func.myCall());  
    }  
}
```

자바에서 제공하는 함수형 인터페이스

- 제네릭 함수형 인터페이스, 기본형 특화 함수형 인터페이스를 제공한다.
- `java.util.function` 패키지에서 제공한다.

● **Function<T, R>**

```
@FunctionalInterface
public interface Function<T,R> {
    R apply(T t) ;
}
```

- **T** 타입 인자를 받아서 **R** 타입을 리턴한다.
- 람다식으로는 **T -> R**로 표현한다.
- 함수 조합용 메소드
 - `andThen`
 - `compose`

andThen 과 compose 의 차이점

구분	andThen	compose
적용 순서	첫 번째 함수 적용 후 두 번째 함수 적용	두 번째 함수 적용 후 첫 번째 함수 적용
메서드 호출	f1.andThen(f2)	f1.compose(f2)
데이터 흐름	f1(입력) -> f2(결과)	f2(입력) -> f1(결과)

```
import java.util.function.Function;

public class App {
    Run | Debug
    public static void main(String[] args) throws Exception {

        // 문자열을 정수로 변환한다.
        Function<String, Integer> stringToInteger = str -> Integer.parseInt(str);

        // 정수에 2를 곱한다
        Function<Integer, Integer> multiplyByTwo = num -> num * 2;

        //andThen
        Function<String, Integer> andThenFunc = stringToInteger.andThen(multiplyByTwo);

        int result = andThenFunc.apply("10");

        System.out.println(result);

        //compose
        Function<String, Integer> composeFunc = multiplyByTwo.compose(stringToInteger);

        result = composeFunc.apply("10");

        System.out.println(result);
    }
}
```


• Consumer<T>

```
@FunctionalInterface  
  
public interface Consumer<T> {  
  
    void accept(T t) ;  
  
}
```

- T 타입 인자 하나를 받고 아무값도 리턴하지 않는다. (값을 소비하는 역할)
- 람다식으로는 T -> void 로 표현한다
- 함수 조합용 메소드

◦ andThen

```
import java.util.function.Consumer;  
  
public class App1 {  
    Run | Debug  
    public static void main(String[] args) {  
        // 람다 표현식을 사용하여 인터페이스를 구현  
        Consumer<String> printMessage = msg -> System.out.println("message : " + msg);  
        printMessage.accept("Hello, Lambda");  
    }  
}
```

● Supplier<T>

```
@FunctionalInterface  
  
public interface Supplier<T> {  
  
    T get( ) ;  
  
}
```

- 아무런 인자를 받지 않고 T 타입의 객체를 리턴한다.
- 입력값 없이 값을 생성하거나 제공하는 역할
- 람다식으로는 () -> T 로 표현한다

```
import java.time.LocalDateTime;  
import java.time.format.DateTimeFormatter;  
import java.util.function.Supplier;  
  
public class App1 {  
    Run | Debug  
    public static void main(String[] args) {  
        // 람다 표현식을 사용하여 인터페이스를 구현  
        Supplier<String> supplier = () -> {  
            DateTimeFormatter formatter = DateTimeFormatter.ofPattern("yyyy-MM-dd HH:mm:ss");  
            return formatter.format(LocalDateTime.now());  
        };  
  
        System.out.println(supplier.get());  
    }  
}
```

● Predicate<T>

```
@FunctionalInterface  
  
public interface Predicate<T> {  
  
    boolean test(T t) ;  
  
}
```

- T 타입의 인자 하나를 받아서 **boolean** 타입을 리턴한다.

- 람다식으로는 T -> boolean 로 표현한다

- 함수 조합용 메소드

◦ And ◦ Or ◦ Negate

```
import java.util.function.Predicate;  
  
public class App1 {  
    Run | Debug  
    public static void main(String[] args) {  
        // 람다 표현식을 사용하여 인터페이스를 구현  
        // 숫자가 짝수인지 판별  
        Predicate<Integer> isEven = num -> num % 2 == 0;  
  
        // 숫자가 100 보다 큰지 판별  
        Predicate<Integer> isGreaterThan100 = num -> num > 100;  
  
        // 숫자가 짝수이면서 100보다 큰지 판별  
        Predicate<Integer> isEvenAndGreaterThan100 = isEven.and(isGreaterThan100);  
  
        System.out.println(isEvenAndGreaterThan100.test(400));  
    }  
}
```

• Comparator<T>

```
@FunctionalInterface  
  
public interface Comparator<T> {  
  
    int compare(T o1, T o2) ;  
  
}
```

- 두 객체 **T**를 비교하여 정렬 순서를 결정하는 함수형 인터페이스
- 주로 **Collections.sort()**나 **Arrays.sort()**, **Stream API**의 **sorted()**와 같은 정렬 관련 메서드에서 사용된다.
- **T** 타입의 인자 두개를 받아서 **int** 타입을 리턴한다.
- 람다식으로는 **(T, T) -> int** 로 표현한다

```
class Person {  
    private String name;  
    private int age;  
  
    public Person(String name, int age) {  
        this.name = name;  
        this.age = age;  
    }  
  
    public String getName() {  
        return name;  
    }  
  
    public int getAge() {  
        return age;  
    }  
  
    @Override  
    public String toString() {  
        return "Person{name='" + name + "', age=" + age + "}";  
    }  
}
```

```
public class App {  
    Run | Debug  
    public static void main(String[] args) throws Exception {  
  
        List<Person> persons = Arrays.asList(  
            new Person(name:"Charlie", age:30),  
            new Person(name:"Alice", age:25),  
            new Person(name:"Bob", age:30),  
            new Person(name:"David", age:20)  
        );  
  
        Comparator<Person> byAge = (p1, p2) -> Integer.compare(p1.getAge(), p2.getAge());  
  
        Collections.sort(persons, byAge);  
  
        persons.forEach( person -> System.out.println(person) );  
  
        Comparator<Person> byName = (p1, p2) -> p1.getName().compareTo(p2.getName());  
  
        Collections.sort(persons, byName);  
  
        persons.forEach( person -> System.out.println(person) );  
    }  
}
```

두 개의 인자를 받는 **Bi** 인터페이스

- 두 개의 타입을 인자로 받을 수 있는 인터페이스이다.

함수형 인터페이스	람다식	추상 메소드
BiPredicate	<code>(T, U) -> boolean</code>	<code>boolean test(T t, U u)</code>
BiConsumer	<code>(T, U) -> void</code>	<code>void accept(T t, U u)</code>
BiFunction	<code>(T, U) -> R</code>	<code>R apply(T t, U u)</code>

● UnaryOperator<T>

- `Function<T, R>`의 특수한 형태로 입력값과 반환값이 모두 같은 타입인 **Function**의 특수한 형태이다.

● BinaryOperator<T>

- 두 개의 입력값(**T**와 **U**)을 받아서 하나의 결과값(**R**)을 반환하는 함수형 인터페이스이다.

```
import java.util.function.BiFunction;
import java.util.function.BiPredicate;
import java.util.function.UnaryOperator;

public class App {
    Run | Debug
    public static void main(String[] args) throws Exception {

        // 첫 번째 숫자가 두 번째 숫자보다 큰지 비교하는 BiPredicate 구현
        BiPredicate<Integer, Integer> isGreaterThan = (num1, num2) -> num1 > num2;

        System.out.println(isGreaterThan.test(1, 3));

        // 두 개의 정수(Integer)를 입력받아 합계를 반환하는 BiFunction 구현
        BiFunction<Integer, Integer, Integer> add = (num1, num2) -> num1 + num2;

        System.out.println(add.apply(1, 3));

        // 문자열(String)을 입력받아 대문자로 변환하는 UnaryOperator 구현
        UnaryOperator<String> toUpperCase = str -> str.toUpperCase();

        System.out.println(toUpperCase.apply("java"));

    }
}
```

람다 표현식 (Lambda Expression)

- 함수를 하나의 식으로 표현한 것이다.

(매개변수, ...) -> { 실행문 }

● 매개변수 리스트

매개변수가 없을 때	()
매개변수가 한 개 일때	(a) 또는 a
매개변수가 여러 개 일때	(a, b)
매개변수 타입은 생략 가능하다. 명시할 수도 있다.	

● 바디 { }

- 여러 줄인 경우에 { } 를 사용한다.

- 한 줄인 경우에는 { } 생략 가능, **return** 도 생략 가능하다.

● 변수 캡처

로컬 변수 캡처

- **final** 이거나 **effective final** 인 경우에만 참조할 수 있다.
- 그렇지 않을 경우 동시성 문제가 생길수 있어서 컴파일러가 방지한다.

effective final

- 자바 8부터 지원하는 기능으로 사실상 **final** 변수이다.
- **final** 키워드를 사용하지 않는 변수를 익명 클래스 구현체 또는 람다에서 참조할 수 있다.

익명 클래스 구현체와 달리 '쉐도잉' 하지 않는다.

- 익명 클래스는 새로 **scope**(유효범위)을 만들지만 람다는 람다를 감싸고 있는 **scope**과 같다.

● 익명(Anonymous) 클래스

```
public class App {  
    private int outerInstanceVariable = 10;  
    public void createAnonymousClass() {  
        int localVariable = 10; // 실질적인 final  
        CustomerInterface<String> func = new CustomerInterface<>() {  
            @Override  
            public String myCall() {  
                System.out.println("this : " + this); // 익명 클래스 객체 자신  
                // localVariable = 20; // Error  
                System.out.println("localVariable : " + localVariable);  
                System.out.println("outerInstanceVariable : " + App.this.outerInstanceVariable);  
                return "Hello, World!!";  
            }  
        };  
        System.out.println(func.myCall());  
    }  
};  
  
Run | Debug  
public static void main(String[] args) {  
    new App().createAnonymousClass();  
}
```

● 람다(Lambda)

```
public class App {  
    private int outerInstanceVariable = 10;  
  
    public void createLambda() {  
        int localVariable = 10; // effectively(실질적인) final  
  
        // 람다가 선언된 바깥 스코프와 동일한 스코프를 공유한다.  
        CustomerInterface<String> func = () -> {  
            System.out.println("this : " + this); // 람다를 감싸고 있는 클래스 인스턴스를 가리킨다.  
            // localVariable = 20; // Error  
            System.out.println("localVariable : " + localVariable);  
            System.out.println("outerInstanceVariable : " + this.outerInstanceVariable);  
            return "Hello, World!!";  
        };  
  
        System.out.println(func.myCall());  
    };  
  
    Run | Debug  
    public static void main(String[] args) {  
        new App().createLambda();  
    }  
}
```

메소드 레퍼런스

- 람다가 기존 메소드 또는 생성자 메소드를 호출하는 거라면, 메소드 레퍼런스를 사용해서 매우 간결하게 표현 할 수 있다.

스태틱 메소드 참조	타입::스태틱 메소드
특정 객체의 인스턴스 메소드 참조	객체레퍼런스::인스턴스 메소드
임의 객체의 인스턴스 메소드 참조	타입::인스턴스 메소드 (Stream API)
생성자 참조	타입:: new

- 스태틱 메소드 참조

```
import java.util.function.Predicate;

public class App {

    // 숫자가 짝수인지 판별하는 스테틱(static) 메서드
    public static boolean isEven(int num) {
        return num % 2 == 0;
    }

    Run | Debug
    public static void main(String[] args) {

        // Predicate<Integer> func = (num) -> num % 2 == 0;

        // 메서드 레퍼런스로 Predicate 사용
        Predicate<Integer> func = App::isEven;

        System.out.println(func.test(5));
    }
}
```

- 특정 객체의 인스턴스 메소드 참조

```
// 문자열을 특정 형식으로 변환하는 클래스

import java.util.function.Function;

class MyStringFormatter {
    private String prefix;

    public MyStringFormatter(String prefix) {
        this.prefix = prefix;
    }

    // 인스턴스 메서드: 접두사를 붙여 문자열을 반환
    public String format(String input) {
        return this.prefix + input;
    }
}

public class App {

    Run | Debug
    public static void main(String[] args) {

        MyStringFormatter formatter = new MyStringFormatter(prefix:"Prefix: ");

        Function<String, String> func = formatter::format;

        System.out.println(func.apply("Hello, World"));

    }
}
```

- 특정 타입의 임의 객체 인스턴스 메소드 참조

```
import java.util.function.Function;

public class App {
    Run | Debug
    public static void main(String[] args) {
        // 자바 람다 표현식을 사용하여 함수형 인터페이스 구현
        Function<String, String> func1 = (str) -> str.toUpperCase();

        // 메소드 레퍼런스
        Function<String, String> func2 = String::toUpperCase;

        System.out.println(func2.apply("java"));
    }
}
```

- 생성자 참조

```
class Person {
    private String name;
    private int age;

    public Person() {
    }

    public Person(String name, int age) {
        this.name = name;
        this.age = age;
    }

    // Getters and Setters 생략

    @Override
    public String toString() {
        return "Person[name : " + name + ", age : " + age + "];"
    }
}

public class App {
    Run | Debug
    public static void main(String[] args) {
        // 디폴트 생성자 참조
        Supplier<Person> func = Person::new;

        Person p = func.get();

        System.out.println(p);

        // 매개변수가 있는 생성자 참조
        BiFunction<String, Integer, Person> func1 = Person::new;

        Person p1 = func1.apply("일길동", 10);

        System.out.println(p1);
    }
}
```

인터페이스 기본 메소드와 스테틱 메소드

● 기본 메소드 (Default Method)

- 인터페이스에 메소드 선언이 아니라 구현체를 제공하는 방법이다.
- 해당 인터페이스를 구현한 클래스를 깨뜨리지 않고 새 기능을 추가할 수 있다.
- 기본 메소드는 구현체가 모르게 추가된 기능이므로 리스크가 있다.
- 컴파일 에러는 아니지만 구현체에 따라 런타임 에러가 발생할 수 있다.
- 반드시 문서화 할 것. (@implSpec 문서화 태그 사용)

※ @implSpec

@implSpec 주석은 해당 메서드와 하위 클래스 사이의 관계를 설명하여, 하위 클래스들이 그 메서드를 상속하거나 **super** 키워드를 이용해 호출할 때 그 메서드가 어떻게 동작하는지를 명확히 인지하고 사용하게 해야 한다.

- **Object**가 제공하는 기능 (**equals**, **hashCode**)는 기본 메소드로 제공할 수 없다.

- 구현체가 재정의해야 한다.
- 인터페이스를 상속받는 인터페이스에서 다시 추상메소드로 변경할 수 있다.
- 인터페이스 구현체가 재정의할 수도 있다.

- 스테틱 (**static**) 메소드

- 해당 타입 관련 헬퍼 또는 유틸리티 메소드를 제공할 때 인터페이스에 스테틱 메소드를 제공할 수 있다.

Java 8 API의 기본 메소드와 스테틱 메소드

• Iterable<T> 기본 메소드

- **Iterable<T>** 인터페이스는 **for-each** 루프를 사용하여 컬렉션을 순회할 수 있게 해주는 기본 인터페이스이다.

- default void forEach(Consumer<? super T> action)
- default Spliterator<T> spliterator()

```
import java.util.ArrayList;
import java.util.List;

public class App1 {

    Run | Debug
    public static void main(String[] args) {

        List<String> names = new ArrayList<>();
        names.add("Alice");
        names.add("Bob");
        names.add("Charlie");

        // 람다 표현식을 사용한 forEach()
        System.out.println("람다 표현식 사용:");
        names.forEach(name -> System.out.println("Hello, " + name));

        System.out.println("\n-----");

        // 메서드 참조를 사용한 forEach()
        System.out.println("메서드 참조 사용:");
        names.forEach(System.out::println);
    }
}
```

• Collection<E> 기본 메소드

- **stream()** 메서드를 통해 **Stream API**를 활용할 수 있게 해준다.

```
• default Stream<E> stream()
• default boolean removeIf(Predicate<? super E> filter)
• default Spliterator<T> spliterator()
```

● **Comparator<T>** 기본 메소드와 스테틱 메소드

- 두 객체의 순서를 정의하는 데 사용되는 함수형 인터페이스다.

```
• reversed()
• thenComparing()
• static reverseOrder() / naturalOrder()
• static nullsFirst() / nullsLast()
• static comparing()
```

스트림 (**Stream**) Java 8 부터 지원

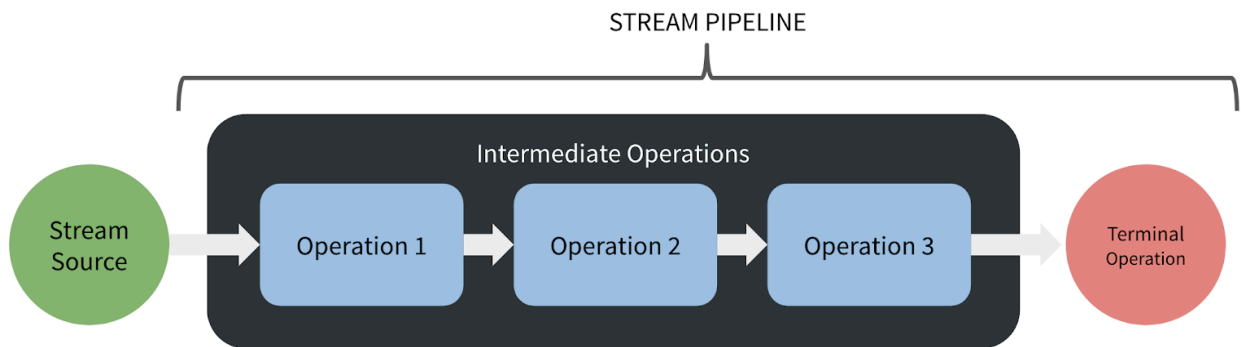
- 데이터 소스(컬렉션, 배열 등)에 저장된 요소를 처리하기 위한 선언적이고 함수형 방식의 추상화이다. 스트림은 데이터 소스로 부터 데이터를 읽기만 할 뿐 원본 데이터를 변경하지 않고 연산을 수행한다.

● 특징

- 지연 연산(**Lazy Evaluation**): 중간 연산(**intermediate operations**)은 스트림 파이프라인이 종료 연산(**terminal operation**)을 만나기 전까지 실행되지 않는다.
- 한 번만 사용 가능: 스트림은 한 번 사용하면 닫히므로 다시 사용하려면 새로운 스트림을 생성해야 한다.
- 병렬 처리 용이: **parallelStream()** 메서드를 통해 병렬 스트림을 쉽게 생성할 수 있다.

● 스트림 파이프라인

- 0 또는 다수의 중개 오퍼레이션 (**intermediate operation**) 과 한 개의 종료 오퍼레이션 (**terminal operation**)으로 구성된다.
- 스트림의 데이터 소스는 오직 터미널 오퍼레이션을 실행할 때만 처리한다.



● Stream<T> 생성 (java.util.stream 패키지)

- 스트림은 다음과 같은 다양한 데이터 소스에서 생성할 수 있다.

- 컬렉션
- 배열
- 가변 매개변수
- 지정된 범위의 연속된 정수
- 특정 타입의 난수
- 람다 표현식
- 파일
- 빈 스트림

1. 컬렉션에서 스트림 생성

```
ArrayList<Integer> list = new ArrayList<Integer>();  
  
Stream<Integer> stream = list.stream();  
  
stream.forEach(System.out::println);
```

2. 배열에서 생성

```
String[] arr = new String[] {"first", "second", "third"};  
  
Stream<String> stream = Arrays.stream(arr);
```

3. 가변 매개변수에서 생성

```
Stream<Double> stream = Stream.of(4.2, 2.5, 3.1, 1.9);  
stream.forEach(System.out::println);
```

<https://ahndding.tistory.com/23> 사이트 참고할 것!!

- **중개 오퍼레이션 (intermediate operation)**

- **Stream**을 리턴한다. 따라서 연속적으로 연결해서 사용할 수 있다.
- **filter, map, limit, skip. sorted ...**

1. 스트림 필터링 : **filter()**

```
Stream<T> filter(Predicate<? super T> predicate)
```

- 스트림에서 주어진 조건(**predicate**)에 맞는 요소만으로 구성된 새로운 스트림을 반환한다.

```
import java.util.Arrays;
import java.util.List;

public class App1 {
    Run | Debug
    public static void main(String[] args) {
        List<String> fruits = Arrays.asList("Apple", "Banana", "Orange", "Grape", "Apple");

        System.out.println("--- 'A'로 시작하는 과일만 필터링 ---");
        fruits.stream()
            .filter(fruit -> fruit.startsWith("A"))
            .forEach(System.out::println);
    }
}
```

2. 스트림 변환 : map(), flatMap()

```
<R> Stream<R> map(Function<? super T, ? extends R> mapper)
```

- 해당 스트림의 요소들을 주어진 함수에 인수로 전달하여 그 반환값으로 이루어진 새로운 스트림을 반환한다. (각 요소를 다른 형태로 변환한다.)

map 메소드

```
import java.util.Arrays;
import java.util.List;

public class App1 {

    Run | Debug
    public static void main(String[] args) {

        List<String> fruits = Arrays.asList("Apple", "Banana", "Orange", "Grape", "Apple");

        System.out.println("--- 모든 과일 이름을 대문자로 변환 ---");

        fruits.stream()
            .map(fruit -> fruit.toUpperCase())
            .forEach(System.out::println);
    }
}
```

flatMap 메소드

```
import java.util.Arrays;
import java.util.stream.Stream;

public class App1 {

    Run | Debug
    public static void main(String[] args) {

        String[] arr = {"I study hard", "You study JAVA", "I am hungry"};

        //하나 이상의 공백으로 분리된 문자열 배열을 다시 스트림을 만들고 이 스트림들을 모두 합쳐서 하나의 단일 리스트로 평탄화한다.
        Arrays.stream(arr)
            .flatMap(s -> Stream.of(s.split(" ")))
            .forEach(System.out::println);

    }
}
```

3. 스트림 제한 : limit(), skip()

//최대 n개의 요소가 담긴 스트림을 반환한다.

```
Stream<T> limit(long maxSize)
```

//앞에서 n개의 요소를 뺀 나머지 스트림을 반환한다.

```
Stream<T> skip(long n)
```

```
import java.util.Arrays;
import java.util.List;

public class App1 {
    Run | Debug
    public static void main(String[] args) {
        List<Integer> numbers = Arrays.asList(1, 2, 3, 4, 5, 6, 7, 8, 9, 10);

        System.out.println("원본 리스트: " + numbers);

        // limit()를 사용하여 처음 5개 요소만 가져오기
        List<Integer> limitedNumbers = numbers.stream()
            .limit(5)
            .collect(java.util.stream.Collectors.toList());

        System.out.println("limit(5) 적용 후: " + limitedNumbers);

        List<String> fruits = Arrays.asList("Apple", "Banana", "Cherry", "Grape", "Orange");

        System.out.println("원본 리스트: " + fruits);

        // skip()을 사용하여 처음 2개 요소를 건너뛰기
        List<String> skippedFruits = fruits.stream()
            .skip(2)
            .collect(java.util.stream.Collectors.toList());

        System.out.println("skip(2) 적용 후: " + skippedFruits);
        // 결과: [Cherry, Grape, Orange]
    }
}
```

4. 스트림 정렬: `sorted()`

```
Stream<T> sorted( )
```

- 스트림의 요소를 정렬하는 데 사용되는 중간 연산(intermediate operation)이다.
- 오름차순으로 정렬된 새로운 스트림을 반환한다.

 **sorted()** : 자연 순서로 정렬

```
import java.util.Arrays;
import java.util.List;
import java.util.stream.Collectors;

public class App {
    Run | Debug
    public static void main(String[] args) throws Exception {

        List<String> colors = Arrays.asList("Red", "Green", "Blue", "Black", "Purple");

        System.out.println("원본 리스트 : " + colors);

        // sorted()를 사용하여 알파벳 순서(자연 순서)로 정렬

        List<String> sortedColors = colors.stream()
            .sorted()
            .collect(Collectors.toList());

        System.out.println("sorted() 적용 후 : " + sortedColors);

    }
}
```

 **sorted(<Comparator<? super T> comparator) :** 커스텀 정렬

```
class Student {

    String name;
    int score;

    public Student(String name, int score) {
        this.name = name;
        this.score = score;
    }

    @Override
    public String toString() {
        return "Student[" + "name='" + name + "', score=" + score + "];"
    }
}

public class App {

    Run | Debug
    public static void main(String[] args) throws Exception {

        List<Student> students = new ArrayList<>();
        students.add(new Student(name:"Alice", score:85));
        students.add(new Student(name:"Bob", score:92));
        students.add(new Student(name:"Charlie", score:85));

        students.sort((s1, s2) -> s1.name.compareTo(s2.name));

        System.out.println("람다 표현식 사용 (이름 오름차순): " + students);
    }
}
```

● 종료 오퍼레이션 (terminal operation)

- Stream을 리턴하지 않는다.
- collect, allMatch, count, forEach, min, max, ...

1. 요소의 출력 : **forEach**

```
void forEach(Consumer<? super T> action)
```

- 스트림의 요소를 반복하여 특정 작업을 수행하는는 중간 연산이다.

```
public class App {  
    Run | Debug  
    public static void main(String[] args) throws Exception {  
        List<String> fruits = Arrays.asList("Apple", "Banana", "Cherry", "Grape");  
  
        // 1. 스트림을 이용해 forEach()로 모든 요소 출력하기  
        fruits.stream()  
            .forEach(fruit -> System.out.println(fruit));  
  
        System.out.println("\n-----");  
  
        fruits.stream()  
            .forEach(System.out::println);  
  
        System.out.println("\n-----");  
  
        // 2. 중간 연산(filter)과 함께 forEach() 사용  
        fruits.stream()  
            .filter(fruit -> fruit.length() > 5)  
            .forEach(System.out::println);  
    }  
}
```

2. 요소의 소모 : **reduce**

- 처음 두 요소를 가지고 연산을 수행한 뒤, 그 결과와 다음 요소를 가지고 또 다시 연산을 수행한다. 이런식으로 해당 스트림의 모든 요소에 대해 연산을 수행하고, 그 결과를 반환한다.

```
T reduce(T identity, BinaryOperator<T> accumulator)
```

```
public class App {  
    Run | Debug  
    public static void main(String[] args) throws Exception {  
        int[] arr = {1, 2, 3, 4, 5};  
        int result = Arrays.stream(arr)  
            .reduce(0, (a, b) -> a + b);  
        System.out.println("totoal : " + result);  
    }  
}
```


3. 요소의 검색: `findFirst()`, `findAny()`

- 스트림을 병렬로 처리할 때 차이가 있다.

- `findFirst()`는 스트림에서 논리적으로 가장 첫 번째 요소를 찾아 반환한다.(순서가 중요)

- `findAny()`는 Multi thread에서 Stream을 처리할 때 가장 먼저 찾은 요소를 리턴한다.

(병렬 스트림에서 성능 최적화가 필요할 때 사용함)

```
public class App2 {  
    Run | Debug  
    public static void main(String[] args) {  
        List<String> list = Arrays.asList("apple", "banana", "cherry", "melon", "watermelon");  
        Optional<String> firstElement = list.stream()  
            .parallel()  
            .filter(s -> s.contains("melon"))  
            .findAny();  
        System.out.println(firstElement.get());  
    }  
}
```

4. 요소의 검사: `anyMatch()`, `allMatch()`, `noneMatch()`

- 스트림 내의 단 하나라도 주어진 **조건**을 만족하는 요소가 있는 경우 `true`를 반환한다.

`boolean anyMatch(Predicate<? super T> predicate)`

- 스트림 내의 모든 요소가 주어진 **조건**을 만족하는 경우 `true`를 반환한다.

`boolean allMatch(Predicate<? super T> predicate)`

- 스트림의 모든 요소가 특정 조건을 만족하지 않을 경우 `true`를 반환한다.

`boolean noneMatch(Predicate<? super T> predicate)`

```
public class App2 {  
    Run | Debug  
    public static void main(String[] args) {  
        List<Integer> numbers = Arrays.asList(1, 3, 5, 8, 9);  
        boolean isEvenNumber = numbers.stream()  
            .anyMatch( num -> num % 2 == 0);  
        System.out.println("isEvenNumber : " + isEvenNumber);  
        boolean allEvenNumber = numbers.stream()  
            .allMatch( num -> num % 2 == 0);  
        System.out.println("allEvenNumber : " + allEvenNumber);  
        boolean noNegativeNumbers = numbers.stream()  
            .noneMatch(num -> num < 0);  
        System.out.println("noNegativeNumbers : " + noNegativeNumbers);  
    }  
}
```

5. 요소의 통계 : **count()**, **min()**, **max()**

```
public class App2 {  
    Run | Debug  
    public static void main(String[] args) {  
        //count()  
        List<String> fruits = Arrays.asList("apple", "banana", "cherry", "strawberry");  
  
        long count = fruits.stream().count();  
  
        System.out.println("count : " + count);  
  
        //min()  
        List<Integer> numbers = Arrays.asList(5, 3, 8, 1, 9);  
  
        Optional<Integer> minNumber = numbers.stream().min(Integer::compare);  
  
        System.out.println(minNumber.get());  
  
        //max()  
        List<String> words = Arrays.asList("apple", "banana", "cherry", "strawberry");  
  
        Optional<String> maxWords = words.stream()  
            .max(Comparator.comparing(String::length));  
  
        System.out.println(maxWords.get());  
    }  
}
```

6. 요소의 연산 : `sum()`, `average()`

-`Stream<T>`는 `sum()`과 `average()` 메서드를 직접 제공하지 않습니다. 이 메서드들을 사용하려면 `mapToInt()`, `mapToLong()`, `mapToDouble()`과 같은 중간 연산자를 사용하여 기본형 특화 스트림으로 변환해야 합니다.

```
public class App2 {  
    Run | Debug  
    public static void main(String[] args) {  
        List<Integer> integers = Arrays.asList(1, 2, 3, 4, 5);  
  
        int sum = integers.stream()  
            .mapToInt(Integer::intValue) // IntStream으로 변환  
            .sum();  
  
        System.out.println("sum : " + sum);  
  
        List<Double> doubles = Arrays.asList(10.5, 20.0, 30.5, 40.0);  
  
        OptionalDouble average = doubles.stream()  
            .mapToDouble(Double::doubleValue) // DoubleStream으로 변환  
            .average();  
  
        System.out.println(average.getAsDouble());  
    }  
}
```

7. 요소의 수집 : `collect()`

- 인수로 전달된 `Collector` 객체에 구현된 방법대로 스트림의 요소를 수집함.
- 스트림의 요소들을 원하는 자료구조(컬렉션)로 변환하는 데 사용된다.

```
<R, A> R collect(Collector<? super T, A, R> collector)
```

Collectors 클래스

자바 8부터 제공되는 `java.util.stream.Collectors` 클래스는 가장 자주 사용되는 `Collector` 구현체들을 정적(static) 메서드로 제공합니다. 이 클래스를 사용하면 복잡한 `Collector`를 직접 구현할 필요 없이 손쉽게 `collect()`를 사용할 수 있습니다

주요 Collectors 메서드와 예시

1.toList()

- 스트림의 모든 요소를 `List`로 수집한다.

```
List<Integer> integers = Arrays.asList(1, 2, 3, 4, 5);  
  
List<Integer> isEvenNumbers = integers.stream()  
    .filter(num -> num % 2 == 0)  
    .collect(Collectors.toList());  
  
System.out.println(isEvenNumbers);
```

2.toSet()

- 스트림의 모든 요소를 Set으로 수집한다. 중복이 제거된다.

```
List<Integer> integers = Arrays.asList(1, 2, 3, 4, 5, 2, 1);

Set<Integer> isEvenNumbers = integers.stream()
    .collect(Collectors.toSet());

System.out.println(isEvenNumbers);
```

3.toMap()

- 스트림의 요소를 키-값 쌍으로 매핑하여 Map으로 수집합니다.

```
List<User> users = new ArrayList<>();
users.add(new User(id:"java1", pwd:"1111", name:"Alice"));
users.add(new User(id:"java2", pwd:"2222", name:"Bob"));
users.add(new User(id:"java3", pwd:"3333", name:"Michale"));
users.add(new User(id:"java4", pwd:"1111", name:"Julia"));

Map<String, User> userMap = users.stream()
    .collect(Collectors.toMap(User::getId, user -> user));

System.out.println(userMap);

User user = userMap.get("java1");
System.out.println(user);
```

4.joining()

- 스트림의 String 요소들을 하나의 문자열로 결합합니다.

```
List<String> words = Arrays.asList("Java", "Stream", "API");  
  
String result = words.stream()  
    .collect(Collectors.joining(","));  
System.out.println(result);
```


Optional<T> (java.util 패키지)

- Java 8 에서 Optional<T> 클래스를 사용해 NullPointerException을 방지할 수 있도록 도와준다.

```
public final class Optional<T> {  
    //If non-null, the value; if null, indicates no value is present  
    private final T value;  
  
    ...  
}
```

- Optional은 리턴 타입으로만 사용하기를 권장한다.
- 메소드의 반환 값이 절대 null이 아니라면 Optional을 사용하지 말자.

Optional은 값을 Wrapping하고 다시 풀고, null일 경우에는 대체하는 함수를 호출하는 등의 오버헤드가 있으므로 잘못 사용하면 성능을 저하시킬 수 있다.

Optional<T> API

● Optional 생성

1. Optional.empty() - 값이 비어있는 경우

```
public static <T> Optional<T> empty()
```

2. Optional.of() - 값이 null이 아닌 경우

```
public static <T> Optional<T> of(T value)
```

3. Optional.ofNullable() - 값이 null이거나 아닌 경우

```
public static <T> Optional<T> ofNullable(T value)
```

● **Optional**에 값이 존재여부 확인

1. `isPresent()`

```
public boolean isPresent()
```

2. `isEmpty()` : Java 11 부터 지원

```
public boolean isEmpty()
```

● **Optional**에 있는 값 가져오기

- 값이 존재하면 값을 반환하고, 그렇지 않은 경우 `NoSuchElementException` 예외를 던진다.

```
public T get()
```

• Optional의 orElse와 orElseGet의 차이

orElse ()

Optional 객체가 값을 가지고 있다면, **orElse()**는 그 값을 반환해요. 하지만 **Optional** 객체가 비어 있다면, **orElse()**에 인수로 전달된 기본값을 반환합니다.

```
public T orElse (T other) {  
    return value != null ? value : other ;  
}
```

orElseGet ()

Optional 객체가 비어 있을 때만 인수로 전달된 **Supplier** 함수를 실행하여 기본값을 생성합니다.

```
public T orElseGet (Supplier<? extends T> other) {  
    return value != null ? value : other.get() ;  
}
```

- 예시 (**orElse**인 경우): 항상 실행

```
public void findByUserEmail (String userEmail) {  
    // orElse에 의해 userEmail이 이미 존재해도 유저 생성 함수가 호출되어 에러가 발생  
    // 단, userEmail이 유니크한 값을 갖는 시스템인 경우  
    return userRepository.findByUserEmail(userEmail)  
        .orElse(createUserWithEmail(userEmail));  
}  
  
private String createUserWithEmail (String userEmail) {  
    User newUser = new User();  
    newUser.setUserEmail(userEmail);  
    return userRepository.save(newUser);  
}
```

- 예시 (**orElseGet**인 경우): 값이 없을 때만 실행

```
public void findByUserEmail (String userEmail) {  
    // orElseGet에 의해 파라미터로 함수가 전달되므로 null이 아니면 유저 생성 함수가  
    // 호출되지 않는다.  
    return userRepository.findByUserEmail(userEmail)  
        .orElseGet(createUserWithEmail(userEmail));  
}  
  
private String createUserWithEmail (String userEmail) {  
    User newUser = new User();  
    newUser.setUserEmail(userEmail);  
    return userRepository.save(newUser);  
}
```

● `orElseThrow()`

- 값이 존재하면 값을 반환하고, 그렇지 않은 경우 특정상황에 맞는 예외를 발생시킬 수 있다.
- `get( )` : `Optional`이 비어 있을 경우 항상 `NoSuchElementException`을 발생시킨다.

```
// 값이 없는 경우
Optional<String> optionalEmpty = Optional.empty();

try {
    optionalEmpty.orElseThrow(() -> new NoSuchElementException("요소를 찾을 수 없습니다.));
} catch (Exception e) {
    System.out.println("Error : " + e.getMessage());
}
```

```
// 값이 있는 경우
Optional<String> optionalWithValue = Optional.of("Hello, World");

System.out.println(optionalWithValue.get());
```

- **filter**

Optional 객체가 값을 가지고 있을 때, 그 값이 특정 조건을 만족하는지 확인하는 데 사용되는 메서드입니다.

```
public Optional<T> filter(Predicate<? super T> predicate)
```

```
// 값이 있는 경우
Optional<Integer> optionalWithValue = Optional.of(15);

Optional<Integer> result = optionalWithValue.filter(n -> n < 10);
System.out.println("result: " + result); // Optional.empty

if (result.isEmpty()) {
    System.out.println("Optional 값은 10 보다 작은 숫자가 아닙니다.");
}
```


Date와 Time API

- 자바 8에서 기존 Date와 Calendar의 문제점을 해소하기 위해서 새로운 날짜와 시간 API가 추가되었다.

- 자바 8에 새로운 **Date**와 **Time API**가 추가된 이유

- 기존에 사용했던 Date와 Calendar 클래스는 **mutable**(변경 가능한) 하기 때문에 스레드에 안전(**thread safe**)하지 않다.
- 클래스 이름이 명확하지 않다. (**Date** 인데 시간까지 다룬다.)
- 버그 발생 여지가 많다. (일관성 없는 요일 상수, 월이 0 부터 시작한다거나 ...)

java.time	날짜와 시간을 다루는데 필요한 핵심 클래스를 제공
java.time.chrono	표준(ISO)이 아닌 달력 시스템을 위한 클래스 제공
java.time.format	날짜와 시간을 파싱, 형식화하기 위한 클래스 제공
java.time.temporal	날짜와 시간의 필드와 단위(Unit)을 위한 클래스 제공
java.time.zone	시간대(time-zone)와 관련된 클래스 제공

※ 타임존이란?

- 지구의 자전에 따라 경도별로 생기는 낮과 밤의 차이를 인위적으로 조정하기 위해서 고안된 시간의 구분선이다.

※ UTC (Coordinated Universal Time/Universal Time Coordinated)란?

- 국제 표준시 (세계 협정시)로 전세계가 공유하고 있는 시간이다.
- 1970년 1월 1일 자정(00:00:00)을 0 밀리초로 설정하여 기준을 삼아 그 후로 시간의 흐름을 밀리초로 계산한다.
- 영국을 기준(UTC+0:00) 으로 각 지역의 시차를 규정한 것이다.
- 서울은 UTC+09:00 이다. (한국은 영국보다 9시간이 빠르다.)
- **timestamp** : 서버의 타임존에 맞춰서 UTC 시간을 표시한다.

예] **timestamp** 형식에 2020-08-25 11:00:00 라고 입력시 현재 타임존이 Asia/Seoul 이라면 2020-08-25 02:00:00 으로 나온다. 출력된 시간은 UTC이기 때문에 2시라고 나왔지만 한국시간으로는 11시를 뜻한다.

Instant vs LocalDateTime

- **LocalDateTime**

- LocalDateTime 클래스는 타임존을 갖지 않는다.
- Local은 타임존, 그리고 오프셋이 없음을 의미한다.

※ **LocalDateTime**은 언제 사용해야 할까?

- 특정 날짜와 시간을 시간대(**timezone**) 정보 없이 표현하기 때문에 여러 위치에 적용하는 경우 사용하면 안 됩니다.

-예를 들어, 한국 서울의 2025년 8월 28일 10:00을 **LocalDateTime**으로 저장하면, 이 값은 항상 **2025-08-28T10:00**으로만 인식됩니다. 만약 이 값을 뉴욕에서 사용하면, 뉴욕 시간으로도 **2025-08-28T10:00**이 되므로, 실제 한국 시간과 13시간 차이가 나는 것을 무시하게 됩니다.

- **Instant**

```
// UTC의 현재 순간을 표시한다.  
  
Instant instant = Instant.now() ;
```

- 시간대와 무관한 특정 시점을 나타냅니다. 이는 '에포크(epoch)'라 불리는 **1970년 1월 1일 00:00:00 UTC**를 기준으로 경과된 나노초를 사용하여 시간을 표현합니다.
- 시스템 로그, 데이터베이스 타임스탬프, 이벤트 기록 등 시간대와 관계없이 절대적인 시점을 저장하고 비교할 때 사용됩니다.

DateTimeFormatter (java.time.format package)

- 날짜 포맷

```
LocalDate date = LocalDate.now();
System.out.println(date);

DateTimeFormatter formatter = DateTimeFormatter.ofPattern("yyyy년 MM월 dd일");
System.out.println(date.format(formatter));
```

● 오전/오후 구하기

```
LocalTime currentTime = LocalTime.now();
System.out.println(currentTime);

DateTimeFormatter formatter = DateTimeFormatter.ofPattern("a HH시 mm분");
System.out.println(currentTime.format(formatter));
```

```
LocalDate now = LocalDate.now();
System.out.println(now);

LocalDate today = LocalDate.of(2022, 4, 1);
System.out.println(today);

System.out.printf("Year : %d%n", now.getYear());
System.out.printf("Month : %s%n", now.getMonth().getValue());
System.out.printf("DayOfMonth : %d%n", now.getDayOfMonth());
System.out.printf("DayOfWeek : %d%n", now.getDayOfWeek().getValue());
System.out.printf("DayOfWeek : %s%n", now.getDayOfWeek());
System.out.printf("isLeapYear : %s%n", now.isLeapYear());

LocalTime currentTime = LocalTime.now();
System.out.println(currentTime);

System.out.printf("Hour : %d%n", currentTime.getHour());
System.out.printf("Minute : %d%n", currentTime.getMinute());
System.out.printf("Second : %d%n", currentTime.getSecond());
```

- 현재 시스템에 설정된 타임 존 구하기

```
System.out.println(ZoneId.systemDefault());
```

Asia/Seoul

타임존 : Asia/Seoul

```
ZoneId zoneId = ZoneId.of("Asia/Seoul");
```

```
ZonedDateTime seoulTime = ZonedDateTime.now(zoneId);
```

```
System.out.println(seoulTime);
```

2022-04-01T16:01:10.617160+09:00[Asia/Seoul]

타임존 : UTC

```
ZoneId utc = ZoneId.of("UTC");
```

```
ZonedDateTime utcTime = ZonedDateTime.now(utc);
```

```
System.out.println(utcTime);
```

2022-04-01T07:01:10.617160Z[UTC]

Nested Interface (중첩 인터페이스)

- 클래스의 멤버로 선언된 인터페이스를 말한다.
- 인터페이스를 클래스 내부에 선언하는 이유는 해당 클래스와 긴밀한 관계를 맺는 구현 클래스를 만들기 위해서이다.

```
public class ClassName {  
    [static] interface InterfaceName {  
        //abstract method  
    }  
}
```

- 클래스 멤버로 선언된 중첩 인터페이스는 **static** 키워드를 붙일수도 있고, 생략도 가능하다.
- 주로 정적 멤버 인터페이스를 많이 사용하는데 **UI** 프로그래밍에서 이벤트를 처리할 목적으로 많이 사용된다.

- 중첩 인터페이스

```
public class Button {  
    OnClickListener listener;  
  
    void setOnClickListener(OnClickListener listener) {  
        this.listener = listener;  
    }  
  
    void touch() {  
        listener.onClick();  
    }  
  
    // 중첩 인터페이스  
  
    static interface OnClickListener {  
        void onClick();  
    }  
}
```

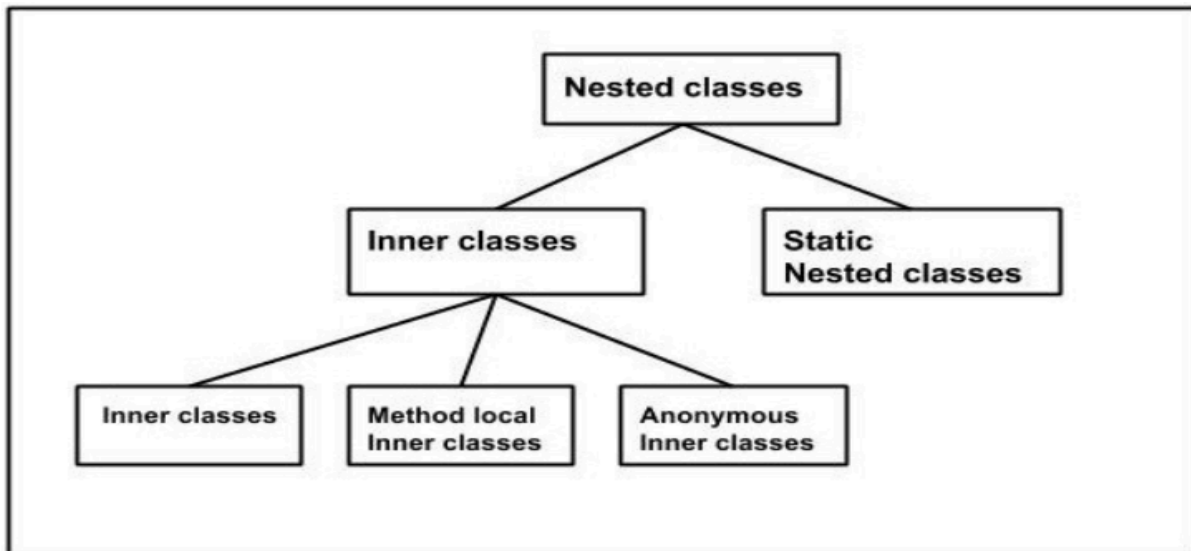
- 구현 클래스

```
public class CallListener implements Button.OnClickListener {  
    @Override  
    public void onClick() {  
        System.out.println("Call CallListener");  
    }  
}  
  
public class MessageListener implements Button.OnClickListener {  
    @Override  
    public void onClick() {  
        System.out.println("Call MessageListener");  
    }  
}
```

- 테스트 클래스

```
public class Test {  
    public static void main(String[] args) {  
        Button btn = new Button();  
  
        btn.setOnClickListener(new CallListener());  
        btn.touch();  
  
        btn.setOnClickListener(new MessageListener ());  
        btn.touch();  
    }  
}
```

Nested Class (중첩 클래스)



- 클래스 내부에 선언한 클래스이다.
- 클래스가 다른 클래스에만 유용 할 경우 (서로간 결합만이 존재하는 클래스의 경우)
해당 클래스에 클래스를 포함시키면 가독성이 좋고 관리하기 편하다.
- 중첩 클래스는 클래스 내부에 숨겨져 있으므로 캡슐화에 도움이 된다.

A와 B라는 클래스가 있다고 가정하자. 여기서 B는 선언 될 A의 멤버에 액세스해야 한다. 클래스 A내에 클래스 B를 숨기면 A의 구성원을 비공개(private)로 선언하고 B가 액세스 할 수 있다. 또한 B 자체는 외부 세계에서 숨길 수 있다.

```
public class Outer {  
    private int x = 10;  
  
    // 중첩 클래스  
    class Inner {  
        int y = 20;  
  
        Inner() { }  
  
        void method1() {  
            // Outer 클래스의 private 선언된 멤버를 참조할 수 있다.  
  
            System.out.println(x);  
  
            System.out.println(y);  
        }  
    }  
  
    public void method2() {  
        // 외부 참조  
  
        Inner inner = new Inner();  
  
        inner.method1();  
    }  
}
```

Nested Class 종류

```
public class ClassName {  
  
    /* 인스턴스 멤버 클래스  
    A 객체를 생성해야만 사용할 수 있음. */  
    class A { }  
  
    /* static 멤버 클래스  
    A 클래스를 바로 접근 가능 */  
    static class B { }  
  
    void method() {  
        /* 로컬 클래스  
        메소드가 실행될 때만 사용할 수 있음. */  
        class C { }  
    }  
}
```

- 인스턴스 멤버 클래스

- **static** 키워드 없이 중첩 선언된 클래스이다.
- 인스턴스 필드와 메소드만 선언이 가능하고 정적 필드와 메소드는 선언할 수 없다.

```
class B {  
    int filed1;  
    class A {  
        A() { }  
        int filed1;  
static int filed2;  
        void method1() {  
            System.out.println(this.field1);  
            System.out.println(B.this.field1);  
        }  
static void method2() { }  
    }  
}
```

- 인스턴스 멤버 클래스의 인스턴스는 바깥 클래스의 인스턴스와 암묵적으로 연결된다. 따라서, 인스턴스 멤버 클래스의 인스턴스 메서드에서 정규화된 **this**를 사용해 바깥 인스턴스의 메서드를 호출하거나 바깥 인스턴스의 참조를 가져올 수 있다. 정규화된 **this**란 클래스명.**this** 형태로 바깥 클래스의 이름을 명시하는 용법을 말한다.
- 인스턴스 멤버 클래스의 인스턴스와 바깥 인스턴스 사이의 관계는 멤버 클래스가 인스턴스화될 때 확립되며, 더 이상 변경할 수 없다. 이 관계는 바깥 클래스의 인스턴스 메서드에서 인스턴스 멤버 클래스의 생성자를 호출할 때 자동으로 만들어지는 게 보통이지만, 드물게는 직접 바깥 인스턴스의 클래스.**new A(args)** 를 호출해 수동으로 만들기도 한다. 이 관계 정보는 인스턴스 멤버 클래스의 인스턴스 안에 만들어져 메모리 공간을 차지하며, 생성 시간도 더 걸린다.
- 외부 참조가 유지된다는 것은 메모리에 대한 참조가 유지되고 있다는 뜻이다. GC가 바깥 클래스의 인스턴스를 수거하지 못하는 메모리 누수가 생길 수 있다.

- 정적(Static) 멤버 클래스

- 내부 클래스가 자신이 바깥 클래스 인스턴스를 참조하지 않는다면 내부 클래스는 `static nested class`로 만드는 것이 낫다.

```
class ClassName {  
    static class A {  
        A() { }  
        int filed1;  
        static int filed2;  
        void method1() { }  
        static void method2() { }  
    }  
}
```

- 로컬 클래스

- 로컬 클래스는 내부에서만 사용되므로 접근을 제한할 필요가 없기 때문에 접근 제한자를 붙일 수 없고 static을 붙일 수 없다.

```
void memthod() {  
    [final] int field = 10; //jdk8부터 final 생략 가능  
    class A {  
        A() {}  
        int field1 = field;  
        static int field2;  
        void method1() { }  
        static void method2() { }  
    }  
    A obj = new A();  
    obj.field1 = 10;  
    obj.method1();  
}
```

- 로컬 클래스는 메소드가 실행될 때 메소드 내에서 객체를 생성하고 사용해야 한다.

- 익명 클래스 (**Anonymous class**)

- 클래스 선언과 객체 생성을 동시에 하는 클래스 이름이 없는 일회용 클래스이다.
- 이름이 없기 때문에 생성자도 가질 수 없으며, 오로지 단 하나의 클래스를 상속받거나 단 하나의 인터페이스만을 구현할 수 있다.

```
new [상위클래스이름 / 구현인터페이스이름] ( ) {  
    // 메소드 구현  
} ;
```